

MDI3003

ADVANCED PREDICTIVE ANALYTICS

LABORATORY MANUAL - EXPERIMENT 05

Product and Brand Sentiment Prediction from Tweet Data

From classical NLP baselines to BiLSTM and Twitter-specific Transformer models

Faculty: Dr. Durgesh Kumar

Assistant Professor (Senior), SCOPE, VIT Vellore

Fall Semester 2026-2027

Manual element	Specification
Syllabus statement	Analysis of tweet data to predict sentiment on a product
Refined laboratory title	Product and Brand Sentiment Prediction from Tweet Data
Core laboratory duration	3 hours
Core learning focus	EDA, tweet preprocessing, TF-IDF, baseline/classical classifiers, multiclass evaluation, error analysis
Advanced extension	BiLSTM, DistilBERT/BERT, BERTweet/Twitter-RoBERTa, cross-dataset transfer and robustness
Recommended environment	Python 3.10+; Google Colab/Jupyter; GPU recommended only for advanced Transformer extension
Primary labels	Positive / Neutral / Negative when using 3-class datasets; binary Positive / Negative for Sentiment140

Student Name: _____ Registration No.: _____

Date: _____ Batch: _____ Instructor Check-off: _____

Revision History

Version	Date	Description
1.0	August 2026	Complete Lab 05 manual with verified public datasets, classical NLP core, advanced BiLSTM/Transformer extension, reproducibility, and responsible social-media analytics.
2.0	August 2026	Incorporated international-benchmark feedback: product-aligned core dataset, lighter 3-hour scope, label harmonization, uncertainty/efficiency reporting, challenge subset, minimum-support entity analysis, and temporal validity.

How to Use This Manual

- Complete the 3-hour core in the stated order. The core is intentionally limited to a reproducible text-classification workflow so that analysis and interpretation are not replaced by rushed model training.
- Use one instructor-approved dataset for the core. The remaining public datasets are used for replication, transfer, or advanced-study experiments.
- Never merge datasets with different label definitions without an explicit harmonization rule. Report every mapping and its limitations.
- Treat usernames, tweet IDs, locations, and other identifying metadata as non-predictive by default. Use tweet text and only justified product/entity fields.
- Do not repeatedly inspect the locked test set. All model and threshold selection must use training/validation evidence.
- Advanced learners may fine-tune a Transformer only after establishing strong classical baselines and a reproducible evaluation pipeline.

Contents

- 1. Laboratory positioning
- 2. Aim and objectives
- 3. Learning outcomes
- 4. Problem statement
- 5. Business and analytical context
- 6. Scope, core/extension boundary, and assumptions
- 7. Verified public datasets
- 8. Dataset schema and governance
- 9. Theoretical background
- 10. Models and representations
- 11. End-to-end workflow
- 12. Software and hardware requirements
- 13. Three-hour plan and instructor check-offs
- 14. Step-by-step laboratory procedure
- 15. Evaluation metrics
- 16. Product/entity-focused analysis
- 17. Error, uncertainty, and robustness analysis
- 18. Visualization requirements
- 19. Core and advanced experiments
- 20. Result-table templates
- 21. Advanced deep-learning and Transformer extension
- 22. Discussion questions
- 23. Viva questions
- 24. Common mistakes

- 25. Responsible social-media analytics
- 26. Submission guidelines
- 27. Assessment rubric
- 28. Final checklist
- 29. Student help and verified references
- 30. Appendices: executable code and acceptance tests

1. Laboratory Positioning

This laboratory treats tweet sentiment analysis as a supervised text-classification problem. Students predict the sentiment expressed toward a named product, brand, service, or entity from tweet text. The core workflow uses transparent lexical and sparse-text baselines; advanced learners compare sequence and Transformer representations after the baseline is established.

Technical positioning: sentiment labels are dataset-defined. A model must not infer an arbitrary “product sentiment” target from unlabeled tweets unless the instructor provides a valid labeling procedure or a labeled public dataset.

2. Aim and Objectives

Aim: To develop, compare, evaluate, and interpret sentiment classifiers for tweets concerning products, brands, services, or named entities, using reproducible NLP pipelines and appropriate public datasets.

3. Learning Outcomes

After completing the laboratory, students should be able to formulate tweet sentiment as a supervised classification task; audit social-media text; build leakage-safe preprocessing and TF-IDF pipelines; compare lexical, probabilistic, and linear classifiers; interpret macro/weighted F1 and confusion matrices; analyze product/entity-specific errors; and explain when deep-learning or Transformer models are justified.

4. Problem Statement

A company wants to understand how customers discuss a product or brand on social media. Given a tweet and, when available, the corresponding product/brand/entity, predict whether the expressed sentiment is positive, neutral, or negative. The output should support aggregate monitoring and analysis rather than autonomous action against individual users.

5. Business and Analytical Context

Tweet sentiment can support product feedback analysis, campaign monitoring, issue discovery, service-quality tracking, and brand-reputation analysis. However, social-media samples are not representative of all customers, sentiment labels are noisy, sarcasm and code-mixing are difficult, and observed sentiment should not be treated as causal evidence or a complete measure of customer satisfaction.

6. Scope, Core/Extension Boundary, and Assumptions

Core (3 hours): one verified product/entity-focused 3-class tweet dataset (preferred: Twitter US Airline Sentiment), EDA, minimal tweet normalization, Dummy and VADER baselines, TF-IDF + Logistic Regression and LinearSVC, locked-test evaluation, class-wise/error analysis, and product/entity-level interpretation. MultinomialNB is retained as a rapid probabilistic comparison or pre-lab task when time permits.

Advanced extension: standardized replication on TweetEval, reproducibly subsampled Sentiment140 binary study, MultinomialNB/character-TF-IDF comparisons, BiLSTM with trainable embeddings, DistilBERT/BERT, BERTweet or Twitter-RoBERTa fine-tuning, three-seed or bootstrap uncertainty analysis, label-harmonized cross-dataset transfer, challenge-set evaluation for sarcasm/emoji/negation/code-mixing, temporal/domain-shift analysis, and product/entity aggregation with minimum-support rules.

- Identify tweet text, product/entity context, target labels, and metadata that must be excluded to prevent leakage.
- Implement classical text representations and sentiment classifiers using a common split and evaluation protocol.
- Compare model quality against both a trivial baseline and a lexical sentiment baseline.
- Analyze class imbalance, difficult language phenomena, and product/entity-specific failure patterns.
- For advanced learners, compare sparse lexical features with sequence embeddings and pretrained Twitter-language models.

7. Verified Public Datasets

The following datasets were verified as publicly accessible as of August 2026. Students must still record the access date, dataset version, license/usage terms, and any instructor-supplied preprocessing.

Dataset	Purpose/source	Size	Labels	Recommended use	Verified public link
---------	----------------	------	--------	-----------------	----------------------

Dataset	Purpose/source	Size	Labels	Recommended use	Verified public link
D1 - TweetEval Sentiment (standardized benchmark)	Twitter sentiment benchmark unified from SemEval-2017 Task 4	45,615 train; 2,000 validation; 12,284 test	negative / neutral / positive	Secondary benchmark / extension; fixed splits and excellent comparability, but not limited to a single product/entity	https://huggingface.co/datasets/cardiffnlp/tweet_eval
D2 - Twitter US Airline Sentiment (preferred core)	Customer tweets about six U.S. airlines (Feb 2015)	14,640 tweets	negative / neutral / positive	Preferred 3-hour core: direct business/service entity sentiment with airline context; verify leakage-prone metadata before modeling	https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment
D3 - Sentiment140	Large general Twitter corpus collected with distant supervision	1,600,000 training tweets	negative / positive (training set)	Large-scale binary extension; use reproducible subsampling, discuss distant-supervision noise, and keep separate from 3-class comparisons unless harmonized	https://www.kaggle.com/datasets/kazanova/sentiment140
D4 - Twitter Entity Sentiment Analysis (optional product/entity dataset)	Entity-level sentiment where each message is associated with an entity	Public train/validation CSV files	Positive / Negative / Neutral; source also contains relevance handling	Strong fit when the experiment explicitly requires sentiment toward a product/brand/entity	https://www.kaggle.com/datasets/jp797498e/twitter-entity-sentiment-analysis
D5 - Brand Sentiment Analysis Dataset (optional)	Tweets assessed for sentiment and associated brand/product	8,589 training rows; 504 test rows reported on dataset page	positive / negative / neutral	Useful product/brand-specific extension; verify exact schema after download	https://www.kaggle.com/datasets/tusharpaul2001/brand-sentiment-analysis-dataset

7.1 Dataset-selection policy

- Preferred core dataset: D2 Twitter US Airline Sentiment, because it directly supports sentiment analysis toward identifiable service entities (airlines) and therefore matches the experiment statement more closely. D1 TweetEval should be used as the standardized secondary benchmark or extension dataset because it provides fixed splits and strong benchmark comparability but is not product-specific.
- For literal product/brand/entity sentiment, D4 or D5 is also suitable for an instructor-approved extension because entity/brand context is explicit. Verify the exact downloaded schema, label semantics, and permitted use before release to students.

- D3 Sentiment140 is intentionally binary in the commonly used training corpus. Use a fixed reproducible subsample for classroom-scale deep-learning experiments, record the random seed and sample size, and explicitly discuss distant-supervision label noise. Do not compare it numerically with 3-class datasets without a predeclared label-harmonization rule.
- Do not pool tweet text across datasets until duplicate checks, label-definition review, domain analysis, and license/usage constraints are documented.

7.2 Mandatory cross-dataset label-harmonization record

Before any transfer or multi-dataset comparison, students must complete a label-harmonization table that records the original labels, operational meaning, any mapped label, labels excluded from comparison, and the justification. If semantics are not compatible, report the datasets separately rather than forcing a common label space.

Dataset	Original labels	Mapped labels	Excluded/ambiguous labels	Justification
TweetEval	negative / neutral / positive	same	none	Direct 3-class benchmark
Twitter US Airline Sentiment	negative / neutral / positive	same	none	Direct 3-class business/service sentiment
Sentiment140	negative / positive	binary only	neutral not available in training labels	Do not fabricate neutral labels
Other dataset	Instructor verifies	Student documents	Student documents	Required before comparison

8. Dataset Schema and Governance

Field	Role	Core use	Risk/control
tweet_id	Identifier	Exclude from predictors	May enable memorization or re-identification
text	Primary predictor	Required	Treat as user-generated text; preserve meaning-bearing cues
sentiment	Target	Required	Use source-defined mapping only
entity / airline / brand / product	Context feature	Optional, justified	Can improve product-specific modeling but creates domain dependence
username / user	Identifier	Exclude	Personal identifier; not required for sentiment classification
location / timezone / coordinates	Metadata	Exclude by default	Privacy/fairness concern; possible spurious correlation
tweet_created / date	Temporal metadata	Optional analysis	Useful for temporal holdout/drift; avoid leakage
sentiment_confidence	Annotation metadata	Do not use as predictor	Direct target-related leakage
negative_reason	Post-label metadata	Do not use in sentiment classifier	Derived from negative sentiment; leaks target information

8.1 Required dataset card

- Dataset name and canonical/public source
- Version/access date and license/usage note
- Number of rows and label distribution
- Train/validation/test definition
- Duplicate and near-duplicate policy
- Language and domain

- Annotation method and likely label noise
- Fields excluded for privacy or leakage
- Known sampling limitations and temporal coverage

9. Theoretical Background

9.1 Sentiment analysis

Sentiment analysis predicts an affective polarity or class from text. For this lab, the target is usually positive, neutral, or negative toward a product, brand, service, or entity.

9.2 Tweet-specific language

Tweets are short and may contain hashtags, mentions, URLs, emojis, elongations, abbreviations, sarcasm, slang, code-mixing, and product handles. Aggressive cleaning can remove sentiment-bearing evidence.

9.3 TF-IDF

TF-IDF increases the weight of terms that are frequent in a document but not ubiquitous across the corpus. Word and character n-grams often provide strong sparse baselines for short social-media text.

9.4 Naive Bayes

Multinomial Naive Bayes estimates class-conditional token evidence under a conditional-independence assumption. It is fast and suitable for non-negative sparse count/TF-IDF features.

9.5 Logistic Regression

Logistic Regression learns linear decision boundaries from sparse features and yields class probabilities through the softmax/logistic link.

9.6 Linear SVM

LinearSVC optimizes a large-margin objective and is often a strong classifier for high-dimensional sparse text. Its decision scores are not calibrated probabilities.

9.7 VADER lexical baseline

VADER is a rule/lexicon-based sentiment system designed for social-media language. It provides a useful no-training baseline but may fail on context, domain language, sarcasm, and emerging product terminology.

9.8 BiLSTM

A bidirectional LSTM learns sequential context from word embeddings in both directions. It can model order and longer dependencies, but may overfit small tweet datasets.

9.9 BERT-family models

BERT/DistilBERT use pretrained contextual representations and can be fine-tuned for sequence classification. Twitter-specific pretrained models such as BERTweet or Twitter-RoBERTa are usually a more natural advanced comparison because their pretraining domain resembles tweets.

9.10 Macro F1

Macro F1 averages class-wise F1 equally and is therefore important when negative, neutral, and positive classes are imbalanced.

9.11 Key equations

TF-IDF: $\text{tfidf}(t,d) = \text{tf}(t,d) \times \log(N / \text{df}(t))$.

Naive Bayes decision: $\hat{y} = \underset{k}{\text{argmax}} P(C_k) \times \prod_j P(x_j | C_k)$.

Precision = $TP / (TP + FP)$; Recall = $TP / (TP + FN)$; $F1 = 2PR / (P + R)$; Macro F1 = $(1/K) \sum_k F1_k$.

10. Models and Representations

Model	Representation	Core/extension	Purpose
DummyClassifier	No linguistic representation	Core	Trivial majority/stratified baseline
VADER	Lexicon + social-media heuristics	Core	No-training sentiment baseline
MultinomialNB	Word TF-IDF, optionally character TF-IDF	Core optional / pre-lab	Fast probabilistic sparse-text comparison; not required if the 3-hour core is time-constrained
Logistic Regression	Word/character TF-IDF	Core	Strong linear probabilistic baseline
LinearSVC	Word/character TF-IDF	Core	Strong large-margin sparse-text baseline
BiLSTM	Trainable word embeddings + token sequence	Advanced	Sequence-model comparison
DistilBERT / BERT	Subword contextual embeddings	Advanced	Generic pretrained Transformer baseline
BERTweet or Twitter-RoBERTa	Twitter-domain pretrained Transformer	Advanced preferred	Domain-matched Transformer benchmark

10.1 Model-selection rule

Select the core model using validation or cross-validation macro F1, with class-wise recall and stability as secondary evidence. Do not select models using final-test performance. Advanced Transformer models must use the same locked test split or official benchmark split as the classical models.

11. End-to-End Workflow

1. Define the target sentiment and product/entity context.
2. Select one verified dataset and freeze its label mapping.
3. Audit rows, labels, duplicates, nulls, identifiers, and leakage columns.
4. Create/follow the official train-validation-test split.
5. Build minimal tweet normalization without destroying sentiment cues.
6. Establish Dummy and VADER baselines.
7. Train TF-IDF + Logistic Regression and LinearSVC; run MultinomialNB as a pre-lab or rapid optional comparison when assigned.
8. Compare training-only validation/CV results and freeze the selected pipeline.
9. Evaluate once on the locked test set.
10. Inspect class-wise errors and product/entity-specific failures.
11. Predict sentiment for new or held-out product tweets.
12. Advanced: train BiLSTM and Twitter-specific Transformer models using the same evaluation protocol.
13. Document limitations, privacy, sampling bias, and reproducibility artifacts.

12. Software and Hardware Requirements

Component	Core	Advanced
Python	3.10+	3.10+
Notebook	Jupyter or Google Colab	Google Colab GPU recommended
Core libraries	pandas, numpy, scikit-learn, matplotlib, nltk, joblib	same
Deep learning	Not required	TensorFlow/Keras or PyTorch
Transformers	Not required	transformers, datasets, evaluate, accelerate
Hardware	CPU sufficient	GPU strongly recommended for fine-tuning

13. Three-Hour Plan and Instructor Check-offs

Time	Activity	Required evidence
0–20 min	Problem framing, dataset card, label audit	Dataset/version, target map, entity field, excluded leakage/PII fields
20–45 min	EDA and tweet-specific cleaning	Class plot, length plot, null/duplicate summary, minimal-cleaning rationale
45–70 min	Frozen split + Dummy/VADER baselines	Split manifest; baseline results
70–110 min	TF-IDF + Logistic Regression and LinearSVC	Training-only CV/validation comparison; MultinomialNB only if pre-lab/time permits
110–140 min	Model selection + locked-test evaluation	Classification report, confusion matrices, macro/weighted F1
140–165 min	Product/entity error analysis	5–10 inspected failures + per-entity support/recall where valid
165–180 min	New tweet predictions, save/reload, check-off	Saved pipeline, predictions, README notes, responsible-use statement

13.1 Check-offs

- Check-off 1 - Dataset/task validity: verified source, label meaning, product/entity context, duplicate audit, and leakage/PII fields documented.
- Check-off 2 - Reproducible split and baselines: frozen/official split, Dummy + VADER results, and training-only preprocessing confirmed.
- Check-off 3 - Model comparison: Logistic Regression and LinearSVC compared under identical folds; model selected without using the final test set.
- Check-off 4 - Final interpretation: locked-test metrics, 5–10 error cases, entity support threshold, responsible-use statement, and saved/reloaded model artifact.

14. Step-by-Step Laboratory Procedure

14.1 Define the research question

Example: “How accurately can positive, neutral, and negative sentiment toward a product/service entity be predicted from tweet text, and which errors matter most for business interpretation?”

14.2 Load the dataset

Use the instructor-provided local file or a documented dataset loader. Never silently switch to another mirror if the expected schema differs.

14.3 Audit data

Report shape, columns, label counts, nulls, duplicate tweet IDs, duplicate text, representative examples, and annotation metadata.

14.4 Remove leakage and identifiers

Exclude annotation confidence, negative reason, gold-reason, username, coordinates, and direct identifiers unless an explicitly justified analysis requires them.

14.5 Minimal tweet normalization

Normalize whitespace, replace URLs with <URL>, mentions with <USER>, and optionally normalize repeated spaces. Preserve negation, emojis, hashtags, exclamation/question marks, and product handles unless an ablation justifies removal.

14.6 Split strategy

Use official TweetEval splits when D1 is selected. Otherwise create one fixed stratified split, or a train/validation/test split, and save row IDs. All comparative models must use exactly the same split.

14.7 Baselines

Run DummyClassifier and VADER before fitting ML models.

14.8 Classical models

Fit TF-IDF inside Pipeline objects. Logistic Regression and LinearSVC are mandatory learned core models and must use the same CV folds. MultinomialNB is a fast probabilistic comparison that may be completed as pre-lab or within the session if time permits.

14.9 Final evaluation

After model selection, evaluate once on the locked test set. Report macro F1, weighted F1, accuracy, per-class metrics, and confusion matrices.

14.10 Product/entity analysis

Where an entity/airline/brand field exists, stratify errors and sentiment distribution by entity. Do not claim that raw tweet proportions represent all customers.

14.11 Save and reload

Save the selected pipeline and verify that reloaded predictions match the original predictions on a small fixed sample.

15. Evaluation Metrics

Metric	Why required	Caution
Accuracy	Overall correctness	Can hide minority-class failure
Macro precision / recall / F1	Equal weight per class	Primary metric for imbalanced 3-class sentiment
Weighted F1	Frequency-weighted summary	Can be dominated by the majority class
Per-class recall	Shows missed positive/neutral/negative tweets	Important for business-risk analysis
Confusion matrix	Reveals error directions	Report counts and row-normalized versions
CV mean $\pm$ SD	Shows stability across folds	Use training data only
Optional bootstrap 95% CI	Quantifies final-test uncertainty	Advanced; do not overinterpret overlapping intervals
Inference latency	Operational comparison	Measure with comparable batch size/hardware

16. Product/Entity-Focused Analysis

- If a dataset contains an entity/brand/airline field, compare sentiment distribution and error rate by entity.

Entity-level reporting rule: do not rank or compare entities from tiny samples. Set a minimum support threshold before inspecting test outcomes (for example, at least 30 held-out tweets per entity, or an instructor-defined threshold). Always report N alongside percentages and add a confidence interval or bootstrap interval for entity-level proportions in the advanced study.

- For a single target product, filter only when the product/entity field or a defensible query definition exists. Do not keyword-filter after seeing test labels.
- Report at least three product/entity examples with representative positive, neutral, and negative tweets after de-identification.
- For multi-entity datasets, assess whether one entity dominates training data or has systematically lower recall.
- Avoid causal language such as “product X caused dissatisfaction” unless the data design supports causal inference.

17. Error, Uncertainty, and Robustness Analysis

Challenge	Inspect?	Expected discussion
Negation (“not good”)	Yes	Give at least one observed example or explain why absent
Sarcasm/irony	Yes	Give at least one observed example or explain why absent
Emoji-only or emoji-heavy sentiment	Yes	Give at least one observed example or explain why absent
Hashtag sentiment (#loveit / #fail)	Yes	Give at least one observed example or explain why absent
Product handle/brand token	Yes	Give at least one observed example or explain why absent
Slang and abbreviations	Yes	Give at least one observed example or explain why absent
Spelling variation and elongation	Yes	Give at least one observed example or explain why absent
Mixed sentiment	Yes	Give at least one observed example or explain why absent
Neutral factual statements	Yes	Give at least one observed example or explain why absent
Quoted/retweeted content	Yes	Give at least one observed example or explain why absent
Code-mixing or non-English text	Yes	Give at least one observed example or explain why absent
Domain-specific terms	Yes	Give at least one observed example or explain why absent

17.1 Confidence and abstention extension

For Logistic Regression or calibrated models, evaluate whether low maximum probability corresponds to higher error. For LinearSVC, use calibrated probabilities only if fitted inside a validation-safe calibration procedure. An optional manual-review rule may be selected from validation data using coverage versus error, not from the final test set.

17.2 Advanced uncertainty requirement

For BiLSTM and Transformer comparisons, report either results from at least three independent random seeds or a justified bootstrap 95% confidence interval on the frozen test set. Do not claim that one advanced model is superior when differences are within normal experimental variability.

17.3 Fixed linguistic challenge subset

Construct or use an instructor-supplied held-out challenge subset containing examples of negation, sarcasm/irony, emoji-heavy sentiment, hashtags, slang/spelling variation, mixed sentiment, and code-mixing where present. This subset must not be used for tuning. Report accuracy/F1 or correct-count by challenge type and discuss failure mechanisms.

17.4 Temporal and platform-validity statement

Record the dataset collection period when known. Explain that older Twitter corpora may not represent current X/social-media language, platform policies, user populations, or product discourse. If timestamps allow, conduct a chronological holdout as an advanced drift experiment.

18. Visualization Requirements

- Class-distribution plot
- Tweet-length distribution
- Top terms or n-grams by class using training data only
- Cross-validation model-comparison plot
- Count confusion matrix
- Row-normalized confusion matrix
- Per-class precision/recall/F1 plot
- Entity/product sentiment distribution when applicable
- Error-rate by entity/product when applicable
- Advanced: BiLSTM learning curves
- Advanced: Transformer training/validation loss or F1
- Advanced: classical vs deep/Transformer performance-runtime plot

19. Core and Advanced Experiments

19.1 Mandatory core experiments

- Dummy baseline versus VADER lexical baseline (mandatory).
- Word unigram TF-IDF + MultinomialNB (recommended rapid probabilistic comparison; pre-lab or in-session if time permits).
- Word unigram/bigram TF-IDF + Logistic Regression (mandatory learned baseline).
- Word and optional character n-gram TF-IDF + LinearSVC (mandatory strong sparse-text baseline).
- One controlled preprocessing ablation (for example preserving vs removing hashtags/emojis).
- Locked-test class-wise error analysis with at least 5–10 inspected examples.
- Product/entity stratification if the chosen dataset contains an entity field.

19.2 Advanced experiments

- Replicate the best classical pipeline on a second verified dataset only after completing the label-harmonization table; do not reuse the original test set for tuning.
- Train a BiLSTM with trainable embeddings and early stopping; repeat with at least three seeds or report justified bootstrap uncertainty.
- Fine-tune DistilBERT or BERT as a generic Transformer baseline and record model identifier, seed, checkpoint, runtime, and resource use.
- Fine-tune BERTweet or Twitter-RoBERTa as the preferred Twitter-domain Transformer baseline; use the same frozen split and uncertainty protocol as other advanced models.
- Compare word TF-IDF, character TF-IDF, BiLSTM, generic Transformer, and Twitter-specific Transformer on predictive quality AND efficiency (training time, inference latency, model size, and approximate memory/resource requirement).
- Evaluate cross-dataset transfer only when label semantics are documented as compatible. Complete the harmonization table first and report domain shift separately from within-dataset results.
- Run product/entity-level coverage/error analysis only for entities meeting the predeclared minimum-support threshold; report N with every percentage and avoid overinterpreting small groups.
- Perform temporal holdout if timestamps are available and discuss sentiment/domain drift.

20. Result-Table Templates

In addition to the existing templates, advanced learners must complete the following two tables.

Dataset	Original label	Mapped label	Compatible?	Reason / caution

Dataset	Source/version	Rows	Classes	Split	License/terms	Primary domain

Model	Representation	CV macro F1 mean	CV macro F1 SD	Weighted F1	Fit time
Dummy					
VADER					
MultinomialNB					
Logistic Regression					
LinearSVC					

Class	Precision	Recall	F1	Support	Most common confusion
Negative					
Neutral					
Positive					

Tweet/case	True	Predicted	Score/confidence	Challenge type	Likely reason	Mitigation

Entity/product	N tweets	% Neg	% Neutral	% Pos	Macro F1	Key caution

Advanced model	Parameters	Macro F1	Weighted F1	Training time	Inference latency	Model size
BiLSTM						
DistilBERT/BERT						
BERTweet/Twitter-RoBERTa						

Advanced efficiency comparison template

Model	Macro F1	Uncertainty	Training time	Inference latency	Model size	Decision/value conclusion
LinearSVC						
BiLSTM						
DistilBERT/BERT						
BERTweet/Twitter-RoBERTa						

21. Advanced Deep-Learning and Transformer Extension

21.1 BiLSTM

Use training-only tokenization/vocabulary fitting, padded sequences, a trainable embedding layer, Bidirectional LSTM, dropout, dense classification head, validation-based early stopping, and a best-checkpoint restore. Report learning curves and explain overfitting if validation performance degrades.

21.2 BERT / DistilBERT

Fine-tune a generic pretrained encoder for sequence classification using the same sentiment labels. Record model identifier, tokenizer, maximum sequence length, batch size, learning rate, epochs, weight decay, seed, and best validation checkpoint.

21.3 BERTweet / Twitter-RoBERTa (preferred advanced comparison)

Because tweets differ from formal text, a model pretrained on Twitter language is a more domain-matched advanced comparison. Use BERTweet or the TweetEval/Twitter-RoBERTa family when available and permitted. Do not claim superiority before evaluating on the same frozen test split.

21.4 Advanced-learning rule

21.5 Sentiment140 scale/noise protocol

For Sentiment140, use a reproducible instructor-defined or seed-controlled subsample when full-corpus training is impractical. Preserve binary labels, document the distant-supervision labeling mechanism and expected noise, and report the exact sample size, seed, class balance, and sampling rule.

21.6 Efficiency and decision-value analysis

Plot macro F1 against training time and, where feasible, model size or inference latency. Students must answer: “Is the observed performance gain large enough to justify the additional compute, memory, complexity, and reproducibility cost?”

A Transformer result is only meaningful if compared against strong classical baselines, trained without test leakage, reported with runtime/resource cost, and accompanied by error analysis. Higher accuracy alone is not a sufficient conclusion.

22. Discussion Questions

14. Why is macro F1 more informative than accuracy when the neutral class dominates?
15. Why should sentiment-confidence or negative-reason fields be excluded from the classifier?
16. What information is lost by removing emojis, hashtags, punctuation, or negation?
17. When does character n-gram TF-IDF help tweets?
18. Why might VADER outperform a weak learned model on a small dataset?
19. Why might LinearSVC outperform MultinomialNB?
20. Why may a generic BERT model underperform a Twitter-specific pretrained model?
21. Why can a BiLSTM overfit a small tweet dataset?
22. How does distant supervision make Sentiment140 labels noisy?

When a Twitter-specific Transformer improves macro F1 only slightly over LinearSVC, what evidence would justify using the larger model in practice?

Why can entity-level sentiment percentages be misleading when entities have very different sample sizes?

Why should results from older Twitter corpora not automatically be interpreted as current X/social-media sentiment?

What must be checked before transferring a model between two tweet datasets that both use labels named positive, neutral, and negative?

23. What makes cross-dataset sentiment transfer difficult?
24. How can sarcasm affect all models?
25. Why is entity/product context sometimes necessary to interpret sentiment?
26. Why should username and location be excluded by default?
27. How can temporal events change sentiment distributions?
28. Why does social-media sentiment not represent the full customer population?
29. What is the difference between predicting sentiment and measuring customer satisfaction?
30. When should a low-confidence tweet be routed for human review?
31. How would you evaluate a model for a new product with few labeled tweets?

23. Viva Questions and Expected Key Points

Question	Expected key point
What is sentiment analysis?	Supervised prediction of affective polarity/class from text.
What are typical labels in this lab?	Positive, neutral, negative; Sentiment140 core training is binary positive/negative.
What is TF-IDF?	Term-frequency weighted by inverse document frequency.
Why use character n-grams?	Robustness to spelling, hashtags, morphology, and noisy short text.
What does Naive Bayes assume?	Conditional independence of features given the class.
Why is VADER a baseline?	No-training social-media lexicon/rule system.
Why use Logistic Regression?	Strong linear sparse-text baseline with probabilities.
Why use LinearSVC?	Large-margin classifier effective in high-dimensional sparse text.
What is macro F1?	Unweighted average of class F1 scores.
What is weighted F1?	Support-weighted average of class F1 scores.
What is data leakage?	Using information unavailable at prediction time or from validation/test during fitting.
Why exclude sentiment confidence?	It is annotation information directly tied to the label.
What is a locked test set?	A final untouched set used after model selection.
What is class imbalance?	Unequal examples per sentiment class.
Why preserve negation?	It can reverse sentiment polarity.
Why preserve emojis?	They can carry strong sentiment signals.
What is an embedding?	Dense vector representation of tokens/text.
What is BiLSTM?	Bidirectional recurrent sequence model using LSTM cells.
What is BERT?	Pretrained bidirectional Transformer encoder.
Why DistilBERT?	Smaller/faster distilled BERT variant.
What is BERTweet?	Transformer pretrained specifically on English tweets.
Why use Twitter-specific pretraining?	Better domain vocabulary/style match.
What is domain shift?	Training and deployment text distributions differ.
What is cross-dataset evaluation?	Testing transfer to another compatible dataset/domain.
Why is sarcasm hard?	Literal words may oppose intended sentiment.
Why avoid user IDs?	No semantic value for generalization and privacy/memorization risk.
What does calibration mean?	Predicted probabilities should correspond to empirical correctness frequency.
What does abstention mean?	Model declines automatic prediction/action for uncertain cases.
Why save a pipeline?	Ensures identical preprocessing at inference and reproducibility.
Why must the test set remain untouched?	To preserve an unbiased estimate of generalization.

24. Common Mistakes

Mistake	Why it is wrong	Correct practice
Randomly merge all public tweet datasets	Different labels/domains/splits create invalid comparisons	Evaluate separately; harmonize only with documented rules
Use annotation confidence or negative reason as features	Leaks target information	Exclude from predictors
Remove all hashtags/emojis/punctuation	Destroys sentiment signals	Use minimal cleaning; test ablations
Fit TF-IDF before splitting	Leaks vocabulary/statistics	Fit inside Pipeline/CV folds
Tune on the test set	Optimistic final performance	Use validation/CV only
Report accuracy only	Hides class-specific failure	Report macro/weighted F1 and class metrics

Mistake	Why it is wrong	Correct practice
Treat Sentiment140 as verified 3-class training data	Its common training set is binary	Map 0/4 to negative/positive; document test differences
Assume all tweets concern a product	General tweet corpora may not be entity-specific	Use entity field or verified product-specific dataset
Train Transformer without baseline	No evidence complexity is justified	Establish classical baselines first
Use BERT and call it Twitter-specific	Generic pretraining differs from tweet-domain pretraining	Compare with BERTweet/Twitter-RoBERTa
Publish raw usernames/coordinates	Privacy and research-ethics risk	Remove identifiers and de-identify examples
Interpret tweet share as market share	Social media is not representative	State sampling limitations explicitly

25. Responsible Social-Media Analytics

Do not infer individual-level traits, protected characteristics, or customer worth from tweet sentiment. Aggregate product/entity summaries must disclose sampling limitations, historical collection period, minimum-support thresholds, and uncertainty. Public availability of a tweet dataset does not eliminate privacy, licensing, or contextual-integrity responsibilities.

- Use only public/licensed or instructor-provided datasets consistent with source terms.
- Minimize personal data: exclude usernames, coordinates, direct identifiers, and unnecessary metadata.
- Do not attempt to re-identify users or contact individuals based on predicted sentiment.
- Do not use sentiment predictions as sole evidence for punitive, discriminatory, employment, credit, or eligibility decisions.
- Document demographic/domain bias, sampling bias, language limitations, and platform-specific behavior.
- Treat dataset labels as annotations, not objective truth.
- Do not infer protected attributes or sensitive personal traits from tweet text for this laboratory.
- When showing examples, redact handles, URLs, addresses, phone numbers, or other identifying details.
- For deployment discussions, require human review, drift monitoring, feedback mechanisms, and periodic revalidation.

26. Submission Guidelines

- RegistrationNumber_Lab05_TweetSentiment.ipynb
- RegistrationNumber_Lab05_Report.pdf
- RegistrationNumber_Lab05_CV_Results.csv
- RegistrationNumber_Lab05_Test_Predictions.csv
- RegistrationNumber_Lab05_Error_Analysis.csv
- RegistrationNumber_Lab05_README.md
- models/selected_pipeline.joblib
- figures/ (all required visualizations)
- advanced/ (only when BiLSTM/Transformer extension is attempted)

The notebook must execute from top to bottom in a clean runtime. The report should be concise (typically 5–7 pages for the core; 7–9 pages for the advanced extension) and should emphasize evidence, error analysis, and limitations rather than screenshots of code.

27. Assessment Rubric - 100 Marks

Criterion	Marks	Excellent evidence	Inadequate evidence
Problem framing and target validity	6	Defines sentiment target, product/entity context, and scope correctly	Vague objective or mismatched labels

Criterion	Marks	Excellent evidence	Inadequate evidence
Dataset provenance and governance	8	Verified source/version, label semantics, product/entity fit, license/usage note, temporal coverage, privacy/leakage audit; harmonization table when multiple datasets are compared	Unverified source or missing governance
EDA and tweet-specific preprocessing	10	Class/length/duplicate analysis; minimal justified normalization	Aggressive or undocumented cleaning
Split design and leakage prevention	10	Official/frozen split; pipelines fit training-only; leakage fields removed	Preprocessing before split or test leakage
Baselines	6	Dummy and VADER both implemented, interpreted, and compared; MultinomialNB reported if assigned/pre-lab	Missing/incorrect baselines
Classical model implementation	15	Leakage-safe Logistic Regression and LinearSVC pipelines use identical split/folds; optional NB comparison is technically valid	Incomplete or incomparable implementations
Model selection and experimental rigor	10	Training-only CV/validation, same folds, defensible selection; advanced claims include multi-seed or bootstrap uncertainty	Model chosen from test results
Final evaluation and visualizations	10	Macro/weighted F1, class metrics, confusion matrices, clear plots	Accuracy-only or weak evidence
Error/product analysis	8	5–10 inspected failures; entity/product patterns and business meaning	Superficial error discussion
Uncertainty/robustness	5	Stability, low-confidence or controlled ablation analysis	No robustness evidence
Responsible analytics	4	Privacy, sampling, bias, misuse boundaries addressed	Responsible-use issues omitted
Reproducibility/artifacts	4	Saved pipeline, versions, manifest, predictions, reload check	Artifacts incomplete
Technical report	4	Concise evidence-based conclusions and limitations	Narrative unsupported by results

Advanced Transformer work may earn instructor-defined bonus/extension credit or replace a subset of core experimentation marks only when the course policy explicitly permits it. The core 100-mark rubric remains achievable without a GPU.

28. Final Submission Checklist

- ☐ Core dataset directly matches product/entity sentiment (preferred D2) or deviation is justified.
- ☐ Cross-dataset label-harmonization table completed before any transfer comparison.
- ☐ Sentiment140, if used, is reproducibly subsampled and its distant-supervision label noise is discussed.
- ☐ Entity/product percentages include sample support; low-support entities are not ranked.

- ☐ Advanced neural/Transformer comparisons report uncertainty and efficiency, not only one best macro F1 value.
- ☐ A fixed linguistic challenge subset is kept out of tuning and analyzed by challenge type when completing the advanced extension.
- ☐ Temporal/platform validity of the tweet corpus is documented.

- ☐ Dataset source, access date, version, labels, and license/usage terms recorded.
- ☐ Core task uses a valid sentiment target.
- ☐ Leakage/annotation fields and identifiers removed.
- ☐ Official/frozen train-validation-test split documented.
- ☐ Dummy and VADER baselines reported.
- ☐ Logistic Regression and LinearSVC compared on the same split; MultinomialNB included when assigned/pre-lab.
- ☐ TF-IDF fitted only inside training/CV pipelines.
- ☐ Macro F1, weighted F1, per-class metrics, and both confusion matrices reported.
- ☐ At least 5–10 errors inspected.
- ☐ Product/entity-specific analysis completed when supported by the dataset.
- ☐ Selected pipeline saved and reload prediction verified.
- ☐ No raw usernames/coordinates exposed in the report.
- ☐ Report limitations include sampling bias, label noise, sarcasm/domain shift.
- ☐ Advanced work, if attempted, records model/tokenizer/checkpoint and GPU/runtime details.
- ☐ Notebook runs top-to-bottom and all artifacts exist.

29. Student Help and Verified References

- [TweetEval GitHub benchmark](#) — Official benchmark repository; sentiment task derives from SemEval-2017 and provides fixed splits.
- [TweetEval on Hugging Face](#) — Convenient dataset access; sentiment config exposes negative/neutral/positive labels.
- [Twitter US Airline Sentiment](#) — 14,640 labeled airline tweets; CC BY-NC-SA 4.0 shown on the dataset page.
- [Sentiment140](#) — 1.6M tweet training corpus with distant-supervision labels; use as binary extension.
- [Twitter Entity Sentiment Analysis](#) — Entity-level tweet sentiment; useful when the experiment explicitly focuses on product/brand/entity sentiment.
- [Brand Sentiment Analysis Dataset](#) — Brand/product-related tweet sentiment dataset for optional extension.
- [scikit-learn text tutorial](#) — Text vectorization, pipelines, classification, evaluation.
- [scikit-learn model selection](#) — Cross-validation and model selection.
- [NLTK VADER](#) — Lexicon-based sentiment analyzer used as a core baseline.
- [Hugging Face text classification](#) — Generic Transformer sequence-classification workflow.
- [BERTweet model card](#) — Twitter-pretrained Transformer for advanced learners.
- [CardiffNLP Twitter-RoBERTa sentiment model](#) — Twitter-domain sentiment model/reference for advanced comparison.
- [TweetNLP](#) — Twitter-specific NLP toolkit and pretrained sentiment models.

30. Appendices - Executable Notebook Scaffold

The following code is a scaffold. Students must adapt configurable file/column names to the selected verified dataset and must not fabricate missing columns.

Appendix A - Core setup, loading, and preprocessing

```
# Core packages
!pip -q install vaderSentiment

import os, re, json, time, platform, joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.dummy import DummyClassifier
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import classification_report, confusion_matrix, f1_score, accuracy_score
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

SEED = 42
np.random.seed(SEED)
OUT = 'lab05_outputs'
os.makedirs(OUT, exist_ok=True)

print('Python:', platform.python_version())

# Configure these after checking the instructor-selected dataset.
DATA_PATH = 'Tweets.csv'
TEXT_COL = 'text'
TARGET_COL = 'airline_sentiment' # change only if dataset schema differs
ID_COL = 'tweet_id'

# Example for Twitter US Airline Sentiment
raw = pd.read_csv(DATA_PATH)
required = {TEXT_COL, TARGET_COL}
missing = required - set(raw.columns)
assert not missing, f'Missing required columns: {missing}'

df = raw[[c for c in [ID_COL, TEXT_COL, TARGET_COL, 'airline'] if c in raw.columns]].copy()
df = df.dropna(subset=[TEXT_COL, TARGET_COL])
df[TEXT_COL] = df[TEXT_COL].astype(str)
print(df.shape)
print(df[TARGET_COL].value_counts(dropna=False))
```

Appendix B - Minimal tweet normalization

```
def normalize_tweet(text):
    text = str(text)
    text = re.sub(r'https?://\S+|www\.\S+', ' <URL> ', text)
    text = re.sub(r'@\w+', ' <USER> ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Do not remove negation, emojis, hashtags, or punctuation by default.
df['clean_text'] = df[TEXT_COL].map(normalize_tweet)

print('Duplicates by ID:', df[ID_COL].duplicated().sum() if ID_COL in df else 'N/A')
print('Duplicate text:', df['clean_text'].duplicated().sum())
```

Appendix C - Fixed split

```
# For TweetEval, prefer its official train/validation/test splits instead of this block.
train_df, test_df = train_test_split(
    df, test_size=0.20, random_state=SEED, stratify=df[TARGET_COL]
)

train_df.to_csv(os.path.join(OUT, 'train_manifest.csv'), index=False)
test_df.to_csv(os.path.join(OUT, 'test_manifest.csv'), index=False)
```

```
X_train, y_train = train_df['clean_text'], train_df[TARGET_COL]
X_test, y_test = test_df['clean_text'], test_df[TARGET_COL]
```

Appendix D - Dummy and VADER baselines

```
dummy = Pipeline([
    ('tfidf', TfidfVectorizer(min_df=2)),
    ('clf', DummyClassifier(strategy='most_frequent', random_state=SEED))
])
dummy.fit(X_train, y_train)
print('Dummy macro F1:', f1_score(y_test, dummy.predict(X_test), average='macro'))

analyzer = SentimentIntensityAnalyzer()
def vader_label(text):
    c = analyzer.polarity_scores(text)['compound']
    if c >= 0.05: return 'positive'
    if c <= -0.05: return 'negative'
    return 'neutral'

# Use VADER only when the dataset labels are exactly negative/neutral/positive.
if set(y_train.unique()) <= {'negative', 'neutral', 'positive'}:
    vader_pred = X_test.map(vader_label)
    print('VADER macro F1:', f1_score(y_test, vader_pred, average='macro'))
```

Appendix E - Classical pipelines and cross-validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

models = {
    'MultinomialNB': Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), min_df=2, max_df=0.98, sublinear_tf=True)),
        ('clf', MultinomialNB(alpha=0.5))
    ]),
    'LogisticRegression': Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), min_df=2, max_df=0.98, sublinear_tf=True)),
        ('clf', LogisticRegression(max_iter=2000, class_weight='balanced', random_state=SEED))
    ]),
    'LinearSVC': Pipeline([
        ('tfidf', TfidfVectorizer(ngram_range=(1,2), min_df=2, max_df=0.98, sublinear_tf=True)),
        ('clf', LinearSVC(class_weight='balanced', random_state=SEED))
    ])
}

rows = []
for name, pipe in models.items():
    scores = cross_validate(
        pipe, X_train, y_train, cv=cv,
        scoring={'macro_f1': 'f1_macro', 'weighted_f1': 'f1_weighted', 'accuracy': 'accuracy'},
        n_jobs=-1, return_train_score=False
    )
    rows.append({
        'model': name,
        'macro_f1_mean': scores['test_macro_f1'].mean(),
        'macro_f1_sd': scores['test_macro_f1'].std(),
        'weighted_f1_mean': scores['test_weighted_f1'].mean(),
        'accuracy_mean': scores['test_accuracy'].mean(),
        'fit_time_mean': scores['fit_time'].mean()
    })

cv_results = pd.DataFrame(rows).sort_values('macro_f1_mean', ascending=False)
print(cv_results)
cv_results.to_csv(os.path.join(OUT, 'cv_results.csv'), index=False)
```

Appendix F - Locked-test evaluation and saving

```
best_name = cv_results.iloc[0]['model']
best_model = models[best_name]
best_model.fit(X_train, y_train)

pred = best_model.predict(X_test)
print('Selected:', best_name)
```

```

print(classification_report(y_test, pred, digits=4))
print('Macro F1:', f1_score(y_test, pred, average='macro'))
print('Weighted F1:', f1_score(y_test, pred, average='weighted'))

pred_df = test_df[[c for c in [ID_COL, TEXT_COL, TARGET_COL, 'airline'] if c in test_df.columns]].copy()
pred_df['prediction'] = pred
pred_df.to_csv(os.path.join(OUT, 'test_predictions.csv'), index=False)

joblib.dump(best_model, os.path.join(OUT, 'selected_pipeline.joblib'))
reloaded = joblib.load(os.path.join(OUT, 'selected_pipeline.joblib'))
assert np.array_equal(reloaded.predict(X_test.iloc[:20]), best_model.predict(X_test.iloc[:20]))

```

Appendix G - Product/entity aggregation

```

# Only if an entity/brand/airline column is present.
ENTITY_COL = 'airline'
if ENTITY_COL in pred_df.columns:
    entity_summary = pd.crosstab(
        pred_df[ENTITY_COL], pred_df['prediction'], normalize='index'
    ).round(3)
    print(entity_summary)
    entity_summary.to_csv(os.path.join(OUT, 'entity_sentiment_distribution.csv'))

```

Appendix H - BiLSTM extension scaffold

```

# GPU optional; extension only.
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from sklearn.preprocessing import LabelEncoder

MAX_WORDS = 30000
MAX_LEN = 80
EMBED_DIM = 128

# Fit tokenizer on TRAINING text only.
tok = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
tok.fit_on_texts(X_train)
Xtr = pad_sequences(tok.texts_to_sequences(X_train), maxlen=MAX_LEN, padding='post', truncating='post')
Xte = pad_sequences(tok.texts_to_sequences(X_test), maxlen=MAX_LEN, padding='post', truncating='post')

le = LabelEncoder()
ytr = le.fit_transform(y_train)
yte = le.transform(y_test)

model = Sequential([
    Embedding(MAX_WORDS, EMBED_DIM),
    Bidirectional(LSTM(64)),
    Dropout(0.4),
    Dense(64, activation='relu'),
    Dropout(0.3),
    Dense(len(le.classes_), activation='softmax')
])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
callbacks = [
    EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True),
    ModelCheckpoint(os.path.join(OUT, 'bilstm_best.keras'), save_best_only=True, monitor='val_loss')
]
hist = model.fit(Xtr, ytr, validation_split=0.15, epochs=10, batch_size=64, callbacks=callbacks)

```

Appendix I - Transformer extension scaffold

```

# Advanced extension. Prefer a Twitter-domain model such as vinai/bertweet-base
# or cardiffnlp/twitter-roberta-base when appropriate.
!pip -q install transformers datasets evaluate accelerate

from datasets import Dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification, TrainingArguments, Trainer

```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import evaluate

MODEL_NAME = 'vinai/bertweet-base' # configurable; record exact model/version used

# Create validation data ONLY from the training partition. Keep X_test/y_test locked.
tr_text, val_text, tr_y_raw, val_y_raw = train_test_split(
    X_train, y_train, test_size=0.15, random_state=SEED, stratify=y_train
)

le = LabelEncoder()
le.fit(y_train)
train_tmp = pd.DataFrame({'text': tr_text.values, 'label': le.transform(tr_y_raw)})
val_tmp = pd.DataFrame({'text': val_text.values, 'label': le.transform(val_y_raw)})
test_tmp = pd.DataFrame({'text': X_test.values, 'label': le.transform(y_test)})

train_ds = Dataset.from_pandas(train_tmp, preserve_index=False)
val_ds = Dataset.from_pandas(val_tmp, preserve_index=False)
test_ds = Dataset.from_pandas(test_tmp, preserve_index=False)

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, use_fast=False)
def tokenize(batch):
    return tokenizer(batch['text'], truncation=True, max_length=128)
train_ds = train_ds.map(tokenize, batched=True)
val_ds = val_ds.map(tokenize, batched=True)
test_ds = test_ds.map(tokenize, batched=True)

model = AutoModelForSequenceClassification.from_pretrained(MODEL_NAME, num_labels=len(le.classes_))
metric = evaluate.load('f1')
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return {'macro_f1': metric.compute(predictions=preds, references=labels, average='macro')['f1']}

args = TrainingArguments(
    output_dir=os.path.join(OUT, 'transformer'),
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=3,
    weight_decay=0.01,
    eval_strategy='epoch',
    save_strategy='epoch',
    load_best_model_at_end=True,
    metric_for_best_model='macro_f1',
    greater_is_better=True,
    seed=SEED,
    report_to='none'
)
trainer = Trainer(model=model, args=args, train_dataset=train_ds, eval_dataset=val_ds,
                  tokenizer=tokenizer, compute_metrics=compute_metrics)
trainer.train()

# Evaluate the locked test set exactly once after model selection.
test_metrics = trainer.evaluate(test_ds)
print(test_metrics)

```

Appendix J - Acceptance tests

```

assert TEXT_COL in df.columns and TARGET_COL in df.columns
assert df[TARGET_COL].nunique() >= 2
assert not df[TEXT_COL].isna().any()
assert len(set(train_df.index).intersection(set(test_df.index))) == 0
assert all(name in models for name in ['LogisticRegression', 'LinearSVC'])
assert os.path.exists(os.path.join(OUT, 'cv_results.csv'))
assert os.path.exists(os.path.join(OUT, 'selected_pipeline.joblib'))
assert os.path.exists(os.path.join(OUT, 'test_predictions.csv'))
print('Core acceptance tests passed.')

```

Appendix K - Instructor Setup Notes

- Choose and publish the core dataset before the lab; do not let groups silently use different label definitions.
- For TweetEval, distribute or demonstrate the official split loader.
- For Kaggle datasets, provide either an instructor-downloaded copy or clear access instructions because student accounts/network restrictions may vary.
- If GPU access is unreliable, keep Transformer work as post-lab extension and supply a small stratified training subset for demonstration only.
- Verify all external links shortly before delivery because hosting, licenses, and package APIs can change.
- If using real tweet examples in teaching slides/reports, prefer de-identified excerpts and follow the dataset/platform terms.

Quality-Assurance Summary

This manual separates a feasible 3-hour classical NLP core from advanced neural/Transformer work; uses publicly verified tweet sentiment datasets; distinguishes binary and three-class label spaces; prevents annotation-field leakage; emphasizes macro F1 and class-wise error analysis; and includes privacy, reproducibility, and domain-shift safeguards. Final classroom deployment still depends on the instructor selecting one canonical core dataset and freezing its exact schema and access method.